

TD3: Recurrent Neural Nets & Sequential data

Objectives: build LSTM-based recurrent neural networks with Keras

Exercise 1: Stock prediction

Objective: Build a simple LSTM architecture and apply it to time series prediction.

The aim of the exercise is to build a LSTM-based neural network in order to predict Facebook stock price evolution. In this example, only past stock prices are used in order to predict the future price. In reality, many other external information could have been injected to the network to improve its prediction.

1. Download the FB.csv file from Moodle. Load it as a Pandas dataframe and analyze it content. If you do not like facebook, you can download another dataset from [yahoo finance](#).
2. Extract the *Open* column and convert it into a list.
3. Use a MinMaxScaler to normalize the data between 0.0 and 1.0.
4. Build the dataset:
 - a. Observations are the prices of the past 60 days (windowed over all the data)
 - b. Target is the next day price

At the point, you should have 2 arrays:

- Xs of shape (2086, 60, 1): 2086 samples of 60 stock values
 - ys of shape (2086): 2086 target price to predict
5. Split the training dataset in order to keep 80% for training and 20% for testing
 6. Build the following sequential model:
 - a. A LSTM layer of 50 units, returns sequences and dropout of 0.2,
 - b. A LSTM layer of 50 units, returns sequences and dropout of 0.2,
 - c. A LSTM layer of 50 units, does not return sequences and dropout of 0.2,
 - d. A Dense layer with 1 neuron and no activation
 7. Compile and train the model with:
 - a. Adam optimizer
 - b. Loss function: mean squared error
 - c. 20 epochs
 - d. Batch size of 32
 8. Plot on the same graph:
 - a. The normalized dataset
 - b. The predicted data from training
 - c. The predicted data from testing

Exercise 2 : NLP-101 - Using LSTM to generate text

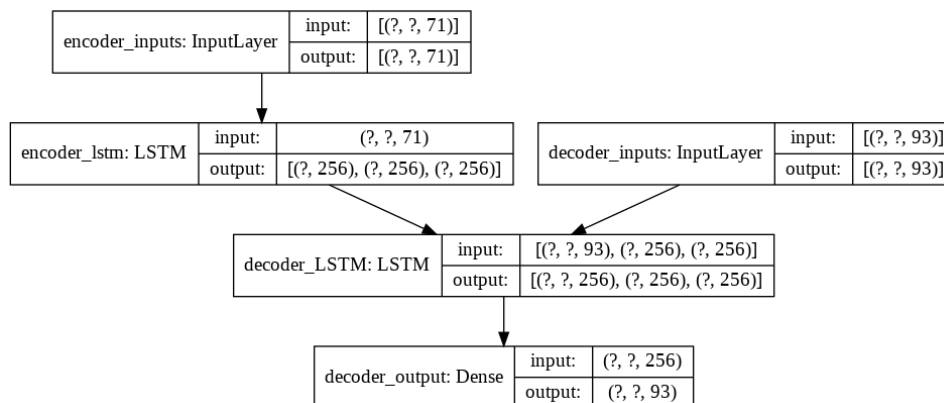
Objective: Build a LSTM-based neural network to generate text on the fly.

In this exercise, we are going to generate sentences (sonnets to be precise). To this end, we will train a deep neural network on *The tragedie of Macbeth* (Shakespeare, 1606). In this case, a char2char generator, the simplest possible use-case, will be developed. Be aware: we will generate old-style english ☺

1. Download the file macbeth.txt from Moodle. Load the file in Python and display its content.
2. From the text, extract each character uniquely in order to build a char <-> int dictionary. Since neural network do not understand character, we need such a system to transform our language and the neural net language (numbers)
3. Prepare the dataset by extracting as much 50-characters-long sequences as samples and the next character as target. Reshape and transform the dataset to be used by a neural network
4. Build a sequential network with the following architecture:
 - a. A LSTM layer with 300 units, dropout of 0.2 and return sequences
 - b. A LSTM layer with 300 units, dropout of 0.2

- c. A dense layer with as many neurons as unique characters in the dataset, activation softmax
5. Compile and train the model with:
 - a. Adam optimizer,
 - b. A categorical crossentropy loss function,
 - c. 100 epochs
 - d. Batch size of 30
6. Use a random seed from the data set to generate a 50-characters-long sequence of characters.
1. the previously predicted sequence and the encoder state vectors.

1. Download the file *fra.txt* from Moodle. Load the file and display some examples.
2. From the text, extract input/target couples that contains english sequences (inputs) and french ones (targets). Build in the same time the set of unique characters for both english and french sequences.
3. Build the char2int and int2char dictionaries for both the input and the target sequences.
4. Generate the training dataset:
 - a. Two Xs vectors that contains the int values of the sequences to encode and to decode
 - b. One ys vector that contains the target sequence to be outputted by the decoder
5. Build the following Encoder-Decoder architecture:
 - a. A LSTM layer (encoder) of 256 units which states are connected to a second LSTM layer (decoder)
 - b. A LSTM layer (decoder) of 256 units which outputs are connected to the next dense layer
 - c. A Dense layer with as many neurons as there are number of characters in the target dictionary, with a softmax activation function.



6. Compile and train the model with the following parameters:
 - a. Adam optimizer with a learning rate of 0.01, beta_1 of 0.9, beta_2 of 0.999 and a decay of 0.001
 - b. Categorical crossentropy loss
 - c. Batch size of 32
 - d. 50 epochs
 - e. Validation split of 0.2
7. Rebuild a model using the parameters previously trained
8. Define a decoder function that feeds the model with an input sequence:
 - a. Calculate the LSTM states of the encoder
 - b. Generate a sequence of characters using the encoded states and the previously predicted sequence.
9. Evaluate the model with sentences from the dataset.